

ADR-01 — Estratégia de Migração Cloud da FoodFlow

Architecture Decision Record · Ciclo 3: Cloud e Resiliência

Projeto: FoodFlow — Delivery

Status: Proposto

Autor: Gabriel Carvalho — 23201412

Data: 22/05/2026

Disciplina: Arquitetura de Software

Professor: Carlos

Repositório: github.com/GabrielGFC/foodflow-adr

1. Decisão

DECISÃO ARQUITETURAL

Adotar a **Opção A — PaaS Gerenciado**: migrar a aplicação Django para **Railway** e o PostgreSQL para **Supabase**.

2. Contexto

A FoodFlow opera hoje como monolito Python/Django em um VPS único (IaaS manual), com PostgreSQL no mesmo servidor e deploy via SSH. O sistema atingiu o limite operacional:

- **Saturação:** 80% de CPU nos picos de fim de semana — pedidos atrasando.
- **Downtime de deploy:** 30 min/semana, incompatível com SLA >99% (orçamento de 7h12min/mês; um único deploy consome ~7%).
- **Budget rígido:** R\$ 2.000/mês de infra.
- **Time enxuto:** 4 devs, 1 DBA, 1 ops part-time — sem expertise em K8s ou IaC avançada.
- **Janela curta:** a solução precisa entregar valor em semanas, não meses.

Comparação das opções

Critério	A — PaaS	B — IaaS + Docker	C — K8s (EKS)
Custo/mês	~R\$ 800 ✓	~R\$ 1.500	~R\$ 3.500 ✗
Setup	1 semana	3 semanas	2 meses
Controle infra	Baixo	Médio	Total
Auto-scaling	Automático (até 10×)	Manual (script)	Automático granular
Expertise necessária	Baixa	Média	Alta (não temos)
Encaixa no budget?	Sim	Sim	Não

Restrições que pesaram mais

1. **Orçamento (R\$ 2k/mês)** — elimina diretamente a Opção C (+75% acima do teto).
2. **Expertise disponível** — o time não opera K8s/EKS em produção; assumir essa complexidade aumentaria o risco em vez de reduzir.
3. **SLA + janela de entrega** — precisamos reduzir o downtime *agora*, não em 2 meses.

3. Justificativa com Trade-off

✓ O QUE GANHAMOS

- **Custo abaixo do teto:** R\$ 1.200 de folga mensal para APM e tráfego.
- **Time-to-value:** 1 semana vs. 3 semanas (B) ou 2 meses (C).
- **Auto-scaling** até 10 instâncias — resolve o pico do fim de semana.
- **Deploy zero-downtime** nativo — elimina 30 min/semana.
- **PostgreSQL gerenciado:** backup, replicação e patching automáticos.
- **Carga operacional baixa:** ops part-time sustenta sem virar gargalo.

✗ O QUE ABRIMOS MÃO

- **Controle fino de infra:** sem tuning de SO, kernel ou networking.
- **Lock-in:** acoplamento operacional a Railway + Supabase.
- **Granularidade de escala:** escala a aplicação inteira, não por módulo.
- **Custo unitário em escala alta:** PaaS encarece se o volume crescer muito.

Por que o trade-off vale a pena neste contexto

O trade-off central é **controle × velocidade de entrega + carga operacional**. A FoodFlow hoje *não tem* problema de controle fino — tem problema de **saturação, downtime e time pequeno**. Pagar com "menos controle" para resolver os três em uma semana, dentro do orçamento, é uma

troca racional. O lock-in só se torna doloroso em escala muito maior que a atual; até lá, a empresa terá receita e time para reavaliar.

⚠ **Anti-migúé:** K8s (C) é tecnicamente superior em controle e granularidade, mas está **fora do budget** e **acima da capacidade operacional do time**. "Mais moderno" não é justificativa — é custo sem retorno proporcional neste estágio. Docker puro (B) paga 87% a mais que PaaS, ainda exige scripts manuais de auto-scaling e mantém o ops como gargalo: não resolve o problema central.

4. Consequências

Próximos 30 dias

- S1** Provisionar Railway + Supabase; configurar variáveis e secrets; validar deploy de staging.
- S2** Migrar dados do PostgreSQL on-premise para Supabase (dump + restore + validação); ensaiar rollback.
- S3** Cutover em janela de baixo tráfego (madrugada de terça); manter VPS antigo em standby por 7 dias.
- S4** Desligar VPS antigo; instrumentar métricas (latência, erro, throughput) e configurar alertas no Sentry.

Decisões que essa escolha vai forçar no futuro

- **Observabilidade:** PaaS oculta a infra — precisaremos escolher uma stack de APM (Sentry / Datadog / New Relic). Não dá mais para `tail -f` no servidor.
- **Saída do monolito:** quando o auto-scaling do Railway começar a ficar caro (estimativa: $>3\times$ o tráfego atual), seremos forçados a decidir entre extrair módulos críticos para Serverless/Functions ou migrar para K8s com o time já maior.
- **Compliance:** se passarmos a processar pagamentos (PCI-DSS) ou crescer cross-border (LGPD/GDPR), será necessário reavaliar se Supabase atende aos requisitos regulatórios.
- **Disaster Recovery:** documentar como restaurar um backup do Supabase em outro provedor — mitigação parcial do lock-in.

5. Referências

- Aulas de Arquitetura — modelos de serviço (IaaS, PaaS, SaaS, FaaS) e trade-offs cloud.
- Hooker — princípios de decisão arquitetural (contexto sobre tecnologia).
- Caso FoodFlow — Ciclo 1 e enunciado do Ciclo 3.